

LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6



Graph and Tree

Disusun oleh :
Vivaldi Fachmy Fahlevi **NIM.2510817210017**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma Dan Struktur Data Modul 6 : Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Vivaldi Fachmy Fahlevi
NIM : 2510817210017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir.Muhmmad Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
Soal 1 : Konversi Adjacency List ke Adjacency Matrix.....	1
A. Source Code	4
B. Output Program	6
C. Pembahasan	7
Soal 2: Konversi Adjacency Matrix ke Adjacency List.....	9
A. Source Code	11
B. Output Program	13
C. Pembahasan	14
Soal 3: BFS: Jarak Terpendek Keluar Labirin	17
A. Source Code	20
B. Output Program	22
C. Pembahasan	23
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin	26
A. Source Code	29
B. Output Program	31
C. Pembahasan	32
Soal 5: Tree Traversal	35
A. Source Code	37
B. Output Program	46
C. Pembahasan	47

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Soal 1	6
Gambar 2 Screenshot Output Program Soal 2	13
Gambar 3 Screenshot Output Program Soal 3	22
Gambar 4 Screenshot Output Program Soal 4	31
Gambar 5 Screenshot Output Program Soal 5	46

DAFTAR TABEL

Tabel 1. Source Code File prob1.cpp.....	4
Tabel 2 Source Code File prob2.cpp.....	11
Tabel 3 Source Code File prob3.cpp.....	20
Tabel 4 Source Code File prob4.cpp.....	29
Tabel 5 Source Code File RedBlackTree.h.....	37
Tabel 6 Source Code File RedBlackTree.cpp	38
Tabel 7 Source Code File prob5.cpp.....	43

Soal 1 : Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1
U2 V2 W2
. . .
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

```
V1 V2 ... VN
V1 a11 a12 ... a1N
V2 a21 a22 ... a2N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge $A \rightarrow B$ 4 menghasilkan nilai 4 pada baris A kolom B . Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A .

Edge $D \rightarrow C$ 3 menghasilkan nilai 3 pada baris D kolom C . Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

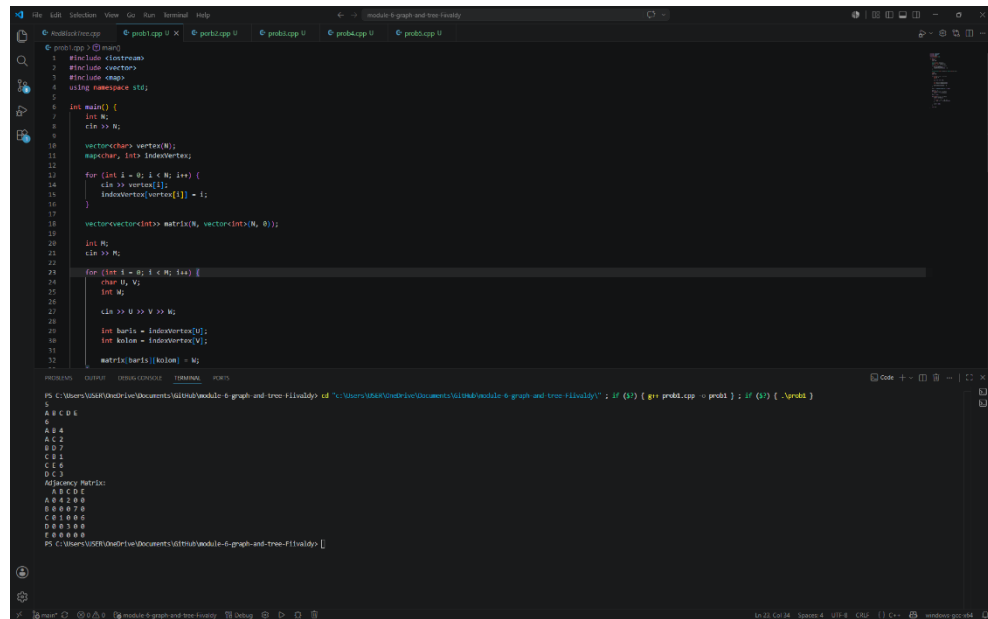
A. Source Code

Tabel 1. Source Code File probl.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <map>
4  using namespace std;
5
6  int main() {
7      int N;
8      cin >> N;
9
10     vector<char> vertex(N);
11     map<char, int> indexVertex;
12
13     for (int i = 0; i < N; i++) {
14         cin >> vertex[i];
15         indexVertex[vertex[i]] = i;
16     }
17
18     vector<vector<int>> matrix(N, vector<int>(N,
19 0));
20
21     int M;
22     cin >> M;
23
24     for (int i = 0; i < M; i++) {
25         char U, V;
26         int W;
27
28         cin >> U >> V >> W;
29
30         int baris = indexVertex[U];
31         int kolom = indexVertex[V];
32
33         matrix[baris][kolom] = W;
34     }
35
36     cout << "Adjacency Matrix:" << endl;
37
38     cout << " ";
39     for (int i = 0; i < N; i++) {
40         cout << " " << vertex[i];
41     }
42     cout << endl;
43
44     for (int i = 0; i < N; i++) {
45         cout << vertex[i];
```

```
46         for (int j = 0; j < N; j++) {
47             cout << " " << matrix[i][j];
48         }
49
50         cout << endl;
51     }
52
53     return 0;
54 }
```

B. Output Program



```
1 #include <iostream>
2 #include <vector>
3 #include <map>
4 using namespace std;
5
6 int main() {
7     int N;
8     cin >> N;
9
10    vector<char> vertex(N);
11    map<char, int> indexVertex;
12
13    for (int i = 0; i < N; i++) {
14        cin >> vertex[i];
15        indexVertex[vertex[i]] = i;
16    }
17
18    vector<vector<int>> matrix(N, vector<int>(N, 0));
19
20    int M;
21    cin >> M;
22
23    for (int i = 0; i < M; i++) {
24        char U, V;
25        int W;
26
27        cin >> U >> V >> W;
28
29        int baris = indexVertex[U];
30        int kolom = indexVertex[V];
31
32        matrix[baris][kolom] = W;
33    }
34
35    // Print the adjacency matrix
36    cout << "Adjacency Matrix:" << endl;
37    for (int i = 0; i < N; i++) {
38        for (int j = 0; j < N; j++) {
39            cout << matrix[i][j] << " ";
40        }
41        cout << endl;
42    }
43
44    return 0;
45 }
```

PS C:\Users\USBR\Documents\4-graph-and-tree-finally> cd "C:\Users\USBR\Documents\4-graph-and-tree-finally" & if (\$?) { g++ prob4.cpp -o prob4 }; if (\$?) { .\prob4 }

5
A B C D E
7
A B C D E
A C 2
B D 7
C D 1
C E 5
D C 3
Adjacency Matrix:
A B C D E
A 0 0 0 0 0
B 0 0 7 0 0
C 0 0 0 0 5
D 0 0 1 0 0
E 0 0 0 0 0

Gambar 1 Screenshot Output Program Soal 1

C. Pembahasan

Kode program pada Soal 1 digunakan untuk membuat *adjacency matrix* dari sebuah graf. Secara sederhana, *adjacency matrix* adalah bentuk tabel yang digunakan untuk menunjukkan hubungan antar simpul atau vertex dalam graf. Pada program ini, setiap vertex disimpan dalam bentuk huruf, misalnya A, B, C, dan seterusnya. Jika terdapat hubungan dari satu vertex ke vertex lain, maka nilai pada matrix akan diisi dengan bobot hubungan tersebut. Jika tidak ada hubungan, maka nilainya tetap 0.

Pada bagian awal program, baris [1–3] digunakan untuk memanggil library yang dibutuhkan. Library *iostream* digunakan agar program dapat menerima input dan menampilkan output. Library *vector* digunakan untuk menyimpan data vertex dan matrix secara dinamis, sedangkan *map* digunakan untuk menghubungkan nama vertex dengan posisi indeksinya di dalam matrix. Dengan adanya *map*, program dapat mengetahui bahwa misalnya vertex A berada pada indeks ke-0, vertex B berada pada indeks ke-1, dan seterusnya.

Pada baris [6–7], program meminta input berupa jumlah vertex yang akan digunakan dalam graf. Nilai tersebut disimpan dalam variabel N. Setelah itu, pada baris [9–10], dibuat sebuah vector bernama *vertex* untuk menyimpan nama-nama vertex, serta *map* bernama *indexVertex* untuk menyimpan pasangan antara nama vertex dan nomor indeksinya. Bagian ini penting karena *adjacency matrix* bekerja berdasarkan posisi baris dan kolom, bukan langsung berdasarkan huruf vertex.

Pada baris [12–15], program melakukan perulangan untuk membaca semua nama vertex yang dimasukkan oleh pengguna. Setiap vertex disimpan ke dalam *vertex[i]*, lalu dicatat juga ke dalam *indexVertex*. Sebagai contoh, jika pengguna memasukkan vertex A, B, dan C, maka program akan menyimpan A pada indeks 0, B pada indeks 1, dan C pada indeks 2. Dengan cara ini, program dapat lebih mudah mencari posisi baris dan kolom dari setiap vertex ketika hubungan antar vertex dimasukkan.

Pada baris [17], program membuat *adjacency matrix* berukuran N x N. Semua nilai awal pada matrix diisi dengan 0. Nilai 0 menunjukkan bahwa pada awalnya belum ada hubungan antar vertex. Misalnya jika terdapat 3 vertex, maka

matrix yang dibuat berukuran 3 baris dan 3 kolom. Baris mewakili vertex asal, sedangkan kolom mewakili vertex tujuan.

Pada baris [19–20], program menerima input jumlah edge atau sisi yang disimpan dalam variabel M. Edge adalah hubungan antar vertex. Setelah itu, pada baris [22–33], program melakukan perulangan sebanyak jumlah edge. Pada setiap perulangan, pengguna memasukkan vertex asal U, vertex tujuan V, dan bobot hubungan W. Bobot ini dapat diartikan sebagai nilai jarak, biaya, waktu, atau ukuran lain yang menunjukkan kekuatan hubungan antar vertex.

Pada baris [28–29], program mencari posisi indeks dari vertex asal dan vertex tujuan menggunakan `indexVertex`. Variabel baris digunakan untuk menyimpan indeks vertex asal, sedangkan variabel kolom digunakan untuk menyimpan indeks vertex tujuan. Setelah posisi baris dan kolom ditemukan, pada baris [31] program mengisi nilai matrix sesuai bobot yang dimasukkan. Misalnya jika terdapat input A B 5, maka artinya ada hubungan dari vertex A menuju vertex B dengan bobot 5. Jika A berada pada baris 0 dan B berada pada kolom 1, maka `matrix[0][1]` akan diisi dengan nilai 5.

Bagian output dimulai pada baris [35]. Program menampilkan teks "Adjacency Matrix:" sebagai judul hasil matrix. Pada baris [37–41], program mencetak bagian kepala kolom matrix, yaitu nama-nama vertex. Hal ini bertujuan agar pembaca dapat mengetahui kolom tersebut mewakili vertex apa. Setelah itu, pada baris [43–52], program mencetak isi matrix secara lengkap. Setiap baris diawali dengan nama vertex, kemudian diikuti oleh nilai-nilai hubungan dari vertex tersebut menuju vertex lainnya.

Program ini menggambarkan graf berarah, karena nilai hubungan hanya diisi pada posisi `matrix[baris][kolom]`. Artinya, jika ada input hubungan dari A ke B, maka yang diisi hanya posisi baris A kolom B. Posisi sebaliknya, yaitu baris B kolom A, tidak otomatis ikut terisi. Dengan demikian, hubungan A ke B dianggap berbeda dengan hubungan B ke A. Jika ingin membuat graf tidak berarah, maka program perlu menambahkan pengisian sebaliknya, yaitu `matrix[kolom][baris] = W`.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
A11 A12 ... A1N
A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1) , (X2,w2) , ...

V2 -> (X1,w1) , (X2,w2) , ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge
- Nilai positif berarti bobot edge
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B, 4) , (C, 2) .

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

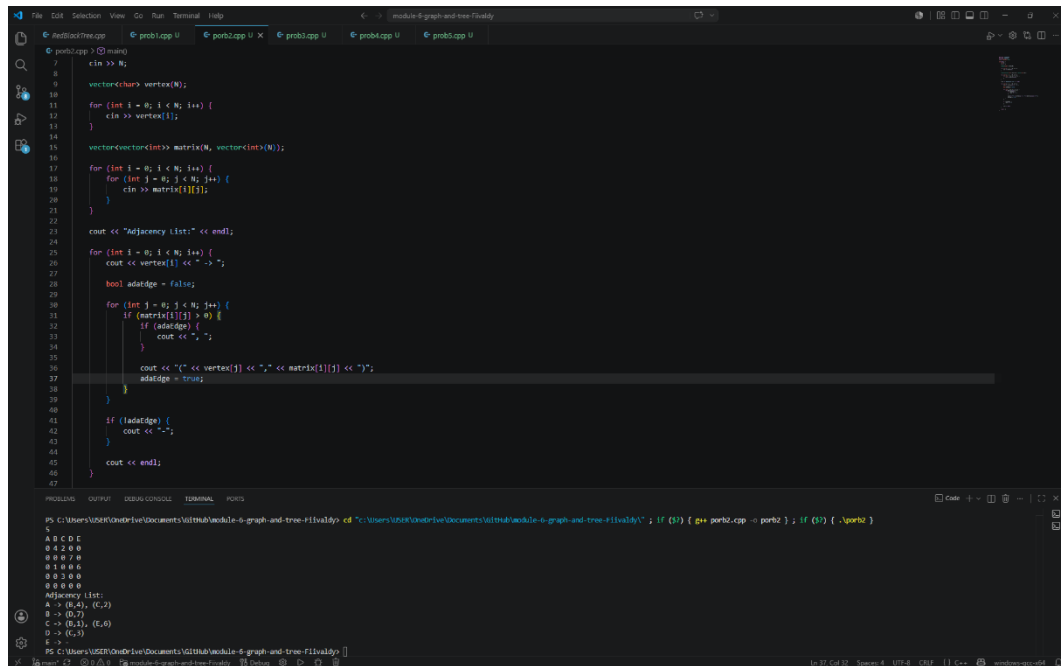
A. Source Code

Tabel 2 Source Code File prob2.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      int N;
7      cin >> N;
8
9      vector<char> vertex(N);
10
11     for (int i = 0; i < N; i++) {
12         cin >> vertex[i];
13     }
14
15     vector<vector<int>> matrix(N, vector<int>(N));
16
17     for (int i = 0; i < N; i++) {
18         for (int j = 0; j < N; j++) {
19             cin >> matrix[i][j];
20         }
21     }
22
23     cout << "Adjacency List:" << endl;
24
25     for (int i = 0; i < N; i++) {
26         cout << vertex[i] << " -> ";
27
28         bool adaEdge = false;
29
30         for (int j = 0; j < N; j++) {
31             if (matrix[i][j] > 0) {
32                 if (adaEdge) {
33                     cout << ", ";
34                 }
35
36                 cout << "(" << vertex[j] << ", " <<
matrix[i][j] << ")";
37                 adaEdge = true;
38             }
39         }
40
41         if (!adaEdge) {
42             cout << "-";
43         }
44
45         cout << endl;
```


46	}
47	
48	return 0;
49	}

B. Output Program



```
PS C:\Users\USER\Documents\GitHub\module-6-graph-and-tree-flivaldy> cd "C:\Users\USER\Documents\GitHub\module-6-graph-and-tree-flivaldy"; if ($?) { g++ port2.cpp -o port2 }; if ($?) { .\port2 }
```

```
5
A B C D E
0 0 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency list:
A -> (B,4), (C,2)
B -> (D,7)
C -> (D,3), (E,6)
D -> (E,5)
E ->
```

Gambar 2 Screenshot Output Program Soal 2

C. Pembahasan

Program Soal 2 berfungsi untuk mengubah data graf dari bentuk **adjacency matrix** menjadi bentuk **adjacency list**. Secara sederhana, adjacency matrix adalah bentuk tabel yang menunjukkan hubungan antar vertex menggunakan baris dan kolom, sedangkan adjacency list menampilkan hubungan tersebut dalam bentuk daftar. Dengan adjacency list, setiap vertex ditampilkan satu per satu, lalu diikuti daftar vertex lain yang terhubung dengannya. Program ini berguna karena bentuk daftar biasanya lebih mudah dibaca ketika ingin melihat hubungan dari satu vertex ke vertex lain secara langsung.

Pada bagian awal, program memanggil library yang diperlukan melalui [1–2]. Library `iostream` digunakan agar program dapat menerima input dan menampilkan output, sedangkan `vector` digunakan untuk menyimpan kumpulan data seperti daftar vertex dan isi matrix. Setelah itu, pada [5–7], program membaca jumlah vertex yang akan digunakan dan menyimpannya ke dalam variabel `N`. Nilai `N` ini menjadi dasar utama dalam program, karena jumlah vertex menentukan banyaknya nama vertex yang akan dibaca serta ukuran matrix yang harus disiapkan. Setelah jumlah vertex diketahui, program membuat tempat penyimpanan bernama vertex pada [9]. Bagian ini digunakan untuk menyimpan nama-nama vertex dalam bentuk karakter, misalnya `A`, `B`, `C`, dan seterusnya. Kemudian pada [11–13], program membaca nama vertex satu per satu sesuai jumlah yang sudah ditentukan. Misalnya jika `N` bernilai 4, maka program akan membaca 4 nama vertex. Penyimpanan nama vertex ini penting karena nantinya nama tersebut akan dipakai saat program menampilkan hasil adjacency list, sehingga output tidak hanya berupa angka indeks, tetapi tetap menggunakan nama vertex yang mudah dipahami.

Pada [15], program membuat matrix dua dimensi menggunakan `vector<vector<int>>`. Matrix ini berukuran $N \times N$, artinya jumlah baris dan kolomnya sama dengan jumlah vertex. Dalam konsep graf, baris pada matrix dapat dipahami sebagai vertex asal, sedangkan kolom sebagai vertex tujuan. Nilai yang berada pada pertemuan baris dan kolom menunjukkan apakah ada hubungan dari vertex asal ke vertex tujuan. Jika nilainya lebih dari 0, berarti ada hubungan dengan bobot tertentu. Jika nilainya 0, berarti tidak ada hubungan langsung.

Bagian [17–21] digunakan untuk membaca isi adjacency matrix dari input. Program membaca nilai matrix satu per satu menggunakan dua perulangan. Perulangan pertama berjalan berdasarkan baris, sedangkan perulangan kedua berjalan berdasarkan kolom. Dengan cara ini, semua nilai hubungan antar vertex dapat dimasukkan secara lengkap. Misalnya, nilai pada baris A kolom B menunjukkan hubungan dari A menuju B. Jika nilainya 5, maka artinya terdapat hubungan dari A ke B dengan bobot 5. Jika nilainya 0, maka tidak ada hubungan langsung dari A ke B.

Setelah semua data matrix berhasil dibaca, program mulai menampilkan hasil dalam bentuk adjacency list pada [23]. Setiap vertex akan dicetak satu per satu melalui perulangan pada [25–45]. Pada awal setiap baris output, program menampilkan nama vertex asal, lalu diikuti tanda panah $\rightarrow$. Tanda panah ini digunakan agar hasil lebih mudah dibaca, karena menunjukkan bahwa daftar setelah tanda panah adalah vertex-vertex tujuan yang terhubung dari vertex tersebut.

Variabel `adaEdge` pada [28] digunakan sebagai penanda apakah suatu vertex memiliki hubungan ke vertex lain atau tidak. Pada awalnya, nilainya dibuat `false`, karena program belum menemukan hubungan apa pun dari vertex yang sedang diperiksa. Setelah itu, pada [30–38], program memeriksa seluruh kolom pada baris matrix tersebut. Jika ditemukan nilai matrix yang lebih dari 0, berarti ada hubungan dari vertex asal ke vertex tujuan. Program kemudian menampilkan vertex tujuan beserta bobotnya dalam bentuk pasangan, misalnya (B,5), yang berarti vertex asal terhubung ke vertex B dengan bobot 5.

Bagian [32–34] digunakan untuk mengatur tanda koma pada output. Jika sebuah vertex memiliki lebih dari satu hubungan, maka setiap hubungan akan dipisahkan dengan koma agar hasilnya terlihat rapi. Program tidak langsung mencetak koma sejak awal, karena hubungan pertama tidak perlu diawali koma. Setelah satu hubungan berhasil dicetak, nilai `adaEdge` diubah menjadi `true` pada [37]. Perubahan ini menandakan bahwa program sudah menemukan setidaknya satu hubungan dari vertex tersebut.

Jika setelah seluruh kolom diperiksa ternyata tidak ada nilai yang lebih dari 0, maka berarti vertex tersebut tidak memiliki hubungan keluar menuju vertex lain. Kondisi ini diperiksa pada [41–43]. Jika `adaEdge` masih bernilai `false`, program

akan mencetak tanda -. Tanda ini digunakan sebagai penanda bahwa vertex tersebut tidak memiliki edge atau hubungan keluar. Dengan begitu, output tetap jelas dan tidak terlihat kosong.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21
G22
...
G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(7, 9)$. Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel $(2, 1)$.

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari $(0, 0)$ ke $(7, 9)$ adalah 16 langkah, sehingga output yang dicetak hanya 16.

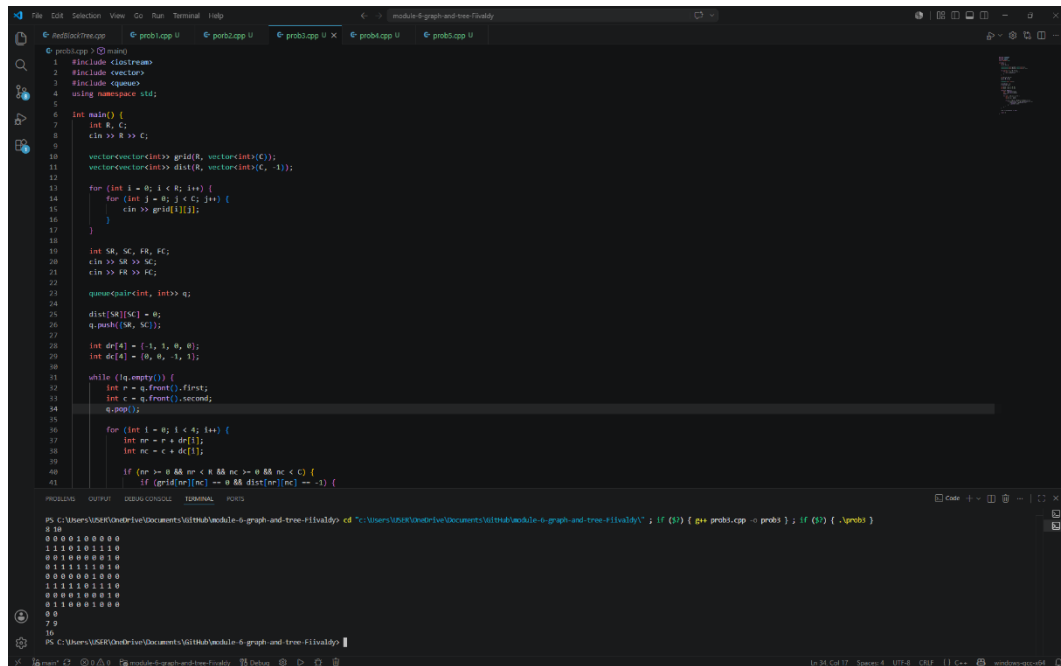
A. Source Code

Tabel 3 Source Code File prob3.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  int main() {
7      int R, C;
8      cin >> R >> C;
9
10     vector<vector<int>> grid(R, vector<int>(C));
11     vector<vector<int>> dist(R, vector<int>(C, -
12 1));
13
14     for (int i = 0; i < R; i++) {
15         for (int j = 0; j < C; j++) {
16             cin >> grid[i][j];
17         }
18     }
19
20     int SR, SC, FR, FC;
21     cin >> SR >> SC;
22     cin >> FR >> FC;
23
24     queue<pair<int, int>> q;
25
26     dist[SR][SC] = 0;
27     q.push({SR, SC});
28
29     int dr[4] = {-1, 1, 0, 0};
30     int dc[4] = {0, 0, -1, 1};
31
32     while (!q.empty()) {
33         int r = q.front().first;
34         int c = q.front().second;
35         q.pop();
36
37         for (int i = 0; i < 4; i++) {
38             int nr = r + dr[i];
39             int nc = c + dc[i];
40
41             if (nr >= 0 && nr < R && nc >= 0 && nc
42 < C) {
43                 if (grid[nr][nc] == 0 &&
44 dist[nr][nc] == -1) {
45                     dist[nr][nc] = dist[r][c] + 1;
46                     q.push({nr, nc});
47                 }
48             }
49         }
50     }
51 }
```

```
44         }
45     }
46 }
47 }
48
49 cout << dist[FR][FC] << endl;
50
51 return 0;
52 }
```

B. Output Program



The screenshot shows a Visual Studio Code editor with a C++ program. The code defines a grid, calculates distances from a source, and uses a priority queue to find the shortest path to a target. The output shows the grid with distances and the path taken.

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 using namespace std;
5
6 int main() {
7     int R, C;
8     cin >> R >> C;
9
10    vector<vector<int>> grid(R, vector<int>(C));
11    vector<vector<int>> dist(R, vector<int>(C, -1));
12
13    for (int i = 0; i < R; i++) {
14        for (int j = 0; j < C; j++) {
15            cin >> grid[i][j];
16        }
17    }
18
19    int SR, SE, FR, FC;
20    cin >> SR >> SE;
21    cin >> FR >> FC;
22
23    queue<pair<int, int>> q;
24
25    dist[SR][SE] = 0;
26    q.push({SR, SE});
27
28    int dr[4] = {-1, 1, 0, 0};
29    int dc[4] = {0, 0, -1, 1};
30
31    while (!q.empty()) {
32        int r = q.front().first;
33        int c = q.front().second;
34        q.pop();
35
36        for (int i = 0; i < 4; i++) {
37            int nr = r + dr[i];
38            int nc = c + dc[i];
39
40            if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
41                if (grid[nr][nc] == 0 && dist[nr][nc] == -1) {
```

Output:

```
PS C:\Users\USER\Documents\GithHub\module-6-graph-and-tree-Flivaldy> cd "C:\Users\USER\Documents\GithHub\module-6-graph-and-tree-Flivaldy"; if ($?) { g++ prob1.cpp -o prob1 }; if ($?) { .\prob1 }
8 10
0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 0
0 0 1 0 0 0 0 1 0
0 1 1 1 1 1 0 1 0
0 0 0 0 0 1 0 0 0
1 1 1 1 0 1 1 1 0
0 0 0 1 0 0 1 1 0
0 1 1 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\USER\Documents\GithHub\module-6-graph-and-tree-Flivaldy>
```

Gambar 3 Screenshot Output Program Soal 3

C. Pembahasan

Program Soal 3 digunakan untuk mencari **jarak terpendek dari titik awal menuju titik tujuan pada sebuah grid**. Grid dapat dipahami sebagai tabel yang terdiri dari baris dan kolom. Setiap kotak pada grid dapat dianggap sebagai posisi yang bisa dilewati atau tidak bisa dilewati. Dalam program ini, nilai 0 pada grid menunjukkan jalan yang dapat dilewati, sedangkan nilai selain 0 dapat dianggap sebagai penghalang. Program menggunakan cara pencarian yang dikenal sebagai **Breadth-First Search (BFS)**, yaitu metode pencarian yang memeriksa posisi terdekat terlebih dahulu sebelum bergerak ke posisi yang lebih jauh. Cara ini cocok digunakan untuk mencari langkah paling pendek pada grid karena setiap perpindahan dihitung dengan jarak yang sama, yaitu satu langkah.

Pada bagian awal program, library yang digunakan dipanggil pada [1–3]. Library `iostream` digunakan untuk menerima input dan menampilkan output. Library `vector` digunakan untuk menyimpan data dalam bentuk tabel, yaitu grid dan jarak. Sementara itu, library `queue` digunakan untuk membuat antrean posisi yang akan diperiksa. Antrean ini penting dalam BFS karena posisi yang lebih dulu ditemukan akan diperiksa lebih dulu, sehingga pencarian berjalan secara teratur dari jarak yang paling dekat.

Pada [6–7], program membaca ukuran grid berupa jumlah baris R dan jumlah kolom C . Nilai ini menentukan besar tabel yang akan digunakan. Misalnya, jika $R = 3$ dan $C = 4$, maka grid memiliki 3 baris dan 4 kolom. Setelah itu, pada [9], program membuat variabel grid untuk menyimpan isi tabel. Kemudian pada [10], dibuat variabel `dist` dengan ukuran yang sama seperti grid. Variabel `dist` digunakan untuk menyimpan jarak dari titik awal ke setiap posisi dalam grid. Semua nilai awal pada `dist` diisi dengan -1, yang berarti posisi tersebut belum pernah dikunjungi atau belum diketahui jaraknya dari titik awal.

Bagian [12–16] digunakan untuk membaca isi grid dari input. Program membaca nilai satu per satu berdasarkan baris dan kolom. Nilai yang dimasukkan ke dalam `grid[i][j]` menunjukkan kondisi suatu kotak. Jika nilainya 0, maka kotak tersebut dapat dilewati. Jika nilainya bukan 0, maka kotak tersebut tidak akan dilewati oleh program. Dengan cara ini, program dapat membedakan mana jalur yang bisa dilalui dan mana yang menjadi penghalang.

Setelah grid dibaca, pada [18–20] program membaca posisi awal dan posisi tujuan. Posisi awal disimpan dalam SR dan SC, yaitu baris dan kolom awal. Posisi tujuan disimpan dalam FR dan FC, yaitu baris dan kolom akhir. Dalam konteks ini, program akan mencari berapa langkah paling sedikit yang dibutuhkan untuk bergerak dari posisi awal menuju posisi tujuan. Posisi tersebut ditulis menggunakan indeks baris dan kolom, sehingga program dapat langsung mengakses kotak yang dimaksud di dalam grid.

Pada [22], program membuat antrean bernama *q* yang berisi pasangan angka. Pasangan angka tersebut digunakan untuk menyimpan posisi dalam bentuk baris dan kolom. Antrean ini menjadi tempat penyimpanan sementara untuk posisi-posisi yang akan diperiksa. Pada [24], jarak posisi awal diatur menjadi 0, karena titik awal tidak membutuhkan langkah untuk sampai ke dirinya sendiri. Setelah itu, pada [25], posisi awal dimasukkan ke dalam antrean agar menjadi posisi pertama yang diperiksa oleh program.

Pada [27–28], program menyiapkan dua array, yaitu *dr* dan *dc*, yang digunakan untuk menentukan arah perpindahan. Program hanya memperbolehkan gerakan ke empat arah, yaitu atas, bawah, kiri, dan kanan. Nilai *dr* digunakan untuk perubahan baris, sedangkan *dc* digunakan untuk perubahan kolom. Misalnya, untuk bergerak ke atas, baris dikurangi satu dan kolom tetap. Untuk bergerak ke bawah, baris ditambah satu dan kolom tetap. Dengan dua array ini, program tidak perlu menulis empat pemeriksaan arah secara terpisah, karena semuanya bisa dilakukan melalui satu perulangan.

Proses utama pencarian dilakukan pada [30–45]. Selama antrean belum kosong, program akan terus mengambil posisi yang berada di bagian depan antrean. Pada [31–33], program mengambil posisi tersebut, menyimpannya ke dalam variabel *r* dan *c*, lalu menghapusnya dari antrean. Posisi ini kemudian dianggap sebagai posisi yang sedang diperiksa. Setelah itu, program melihat kemungkinan gerakan dari posisi tersebut ke empat arah menggunakan perulangan pada [35–44]. Pada [36–37], program menghitung posisi baru yang mungkin dicapai dari posisi saat ini. Posisi baru tersebut disimpan dalam *nr* dan *nc*. Nilai *nr* menunjukkan baris baru, sedangkan *nc* menunjukkan kolom baru. Setelah posisi baru dihitung, program tidak langsung mengunjunginya. Program terlebih dahulu memeriksa

apakah posisi tersebut masih berada di dalam batas grid pada [39]. Pemeriksaan ini penting agar program tidak mencoba mengakses posisi di luar tabel, misalnya baris negatif atau kolom yang lebih besar dari ukuran grid.

Jika posisi baru masih berada di dalam grid, program kemudian memeriksa dua hal pada [40]. Pertama, apakah kotak tersebut bernilai 0, yang berarti dapat dilewati. Kedua, apakah nilai `dist[nr][nc]` masih -1, yang berarti posisi tersebut belum pernah dikunjungi sebelumnya. Dua syarat ini harus terpenuhi agar posisi baru dapat diproses. Jika posisi sudah pernah dikunjungi, program tidak perlu mengunjunginya lagi, karena BFS sudah memastikan bahwa kunjungan pertama adalah jarak terpendek menuju posisi tersebut.

Jika posisi baru memenuhi syarat, maka pada [41] program mengisi jarak posisi baru dengan nilai jarak posisi saat ini ditambah satu. Artinya, untuk sampai ke posisi baru, program membutuhkan satu langkah tambahan dari posisi sebelumnya. Setelah jaraknya diperbarui, posisi baru dimasukkan ke antrean pada [42] agar nantinya juga diperiksa. Dengan cara ini, program secara bertahap menyebar dari titik awal ke posisi-posisi terdekat, lalu ke posisi yang lebih jauh. Pada bagian akhir, program menampilkan nilai `dist[FR][FC]` pada [48]. Nilai ini merupakan jarak terpendek dari titik awal menuju titik tujuan. Jika posisi tujuan dapat dicapai, maka nilai yang ditampilkan adalah jumlah langkah paling sedikit. Namun, jika posisi tujuan tidak dapat dicapai karena terhalang atau tidak ada jalur yang memungkinkan, nilai pada `dist[FR][FC]` tetap -1. Dengan demikian, output -1 berarti tidak ada jalan dari titik awal menuju titik tujuan.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(4, 5)$. Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel $(0, 5)$.

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

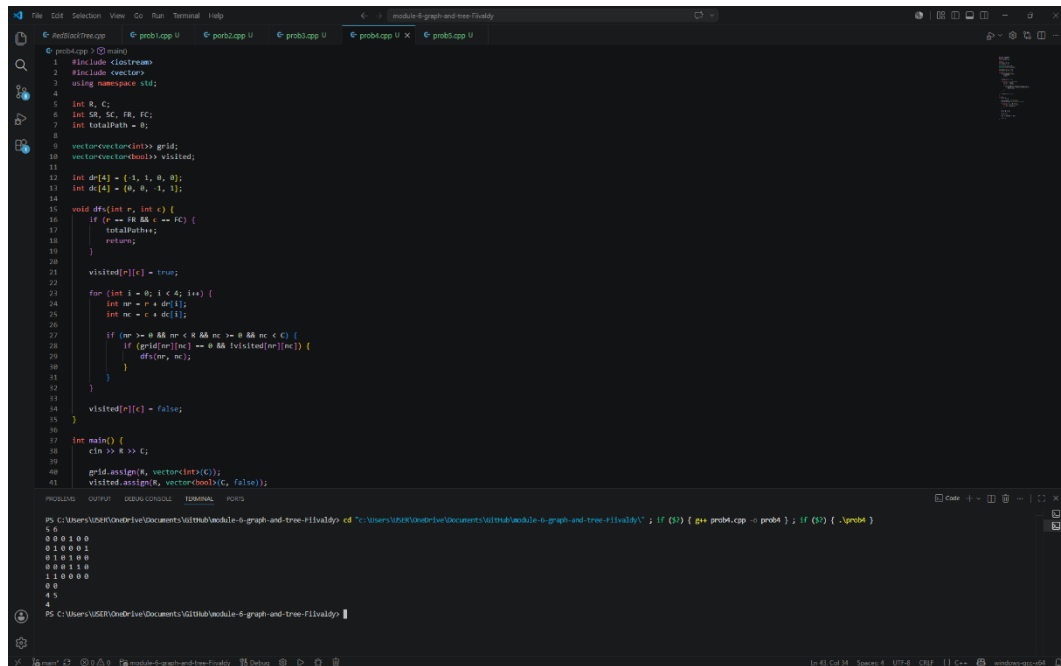
A. Source Code

Tabel 4 Source Code File prob4.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int R, C;
6  int SR, SC, FR, FC;
7  int totalPath = 0;
8
9  vector<vector<int>> grid;
10 vector<vector<bool>> visited;
11
12 int dr[4] = {-1, 1, 0, 0};
13 int dc[4] = {0, 0, -1, 1};
14
15 void dfs(int r, int c) {
16     if (r == FR && c == FC) {
17         totalPath++;
18         return;
19     }
20
21     visited[r][c] = true;
22
23     for (int i = 0; i < 4; i++) {
24         int nr = r + dr[i];
25         int nc = c + dc[i];
26
27         if (nr >= 0 && nr < R && nc >= 0 && nc <
C) {
28             if (grid[nr][nc] == 0 &&
!visited[nr][nc]) {
29                 dfs(nr, nc);
30             }
31         }
32     }
33
34     visited[r][c] = false;
35 }
36
37 int main() {
38     cin >> R >> C;
39
40     grid.assign(R, vector<int>(C));
41     visited.assign(R, vector<bool>(C, false));
42
43     for (int i = 0; i < R; i++) {
44         for (int j = 0; j < C; j++) {
```

```
45         cin >> grid[i][j];
46     }
47 }
48
49     cin >> SR >> SC;
50     cin >> FR >> FC;
51
52     dfs(SR, SC);
53
54     cout << totalPath << endl;
55
56     return 0;
57 }
```

B. Output Program



```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int R, C;
6 int SR, SC, FR, FC;
7 int totalPath = 0;
8
9 vector<vector<int>> grid;
10 vector<vector<bool>> visited;
11
12 int dr[4] = {1, 1, 0, 0};
13 int dc[4] = {0, 0, -1, 1};
14
15 void dfs(int r, int c) {
16     if (r == FR && c == FC) {
17         totalPath++;
18         return;
19     }
20     visited[r][c] = true;
21     for (int i = 0; i < 4; i++) {
22         int nr = r + dr[i];
23         int nc = c + dc[i];
24         if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
25             if (grid[nr][nc] == 0 && !visited[nr][nc]) {
26                 dfs(nr, nc);
27             }
28         }
29     }
30     visited[r][c] = false;
31 }
32
33 int main() {
34     cin >> R >> C;
35     grid.assign(R, vector<int>(C));
36     visited.assign(R, vector<bool>(C, false));
37 }
```

PS C:\Users\User\OneDrive\Documents\12104\module-6-graph-and-tree-flivaldy> cd "C:\Users\User\OneDrive\Documents\12104\module-6-graph-and-tree-flivaldy"; if (\$?) { g++ prob4.cpp -o prob4 }; if (\$?) { .\prob4 }

5 5
0 0 0 1 0 0
0 1 0 0 0 2
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

Gambar 4 Screenshot Output Program Soal 4

C. Pembahasan

Program Soal 4 digunakan untuk menghitung jumlah semua kemungkinan jalur dari titik awal menuju titik tujuan pada sebuah grid. Grid dapat dipahami sebagai tabel yang terdiri dari baris dan kolom. Setiap kotak di dalam grid memiliki nilai tertentu, di mana nilai 0 menunjukkan bahwa kotak tersebut dapat dilewati, sedangkan nilai selain 0 dianggap sebagai penghalang. Berbeda dengan program BFS sebelumnya yang mencari jarak terpendek, program ini tidak mencari jalur paling pendek, tetapi menghitung berapa banyak jalur berbeda yang dapat ditempuh dari titik awal ke titik akhir tanpa melewati kotak yang sama dalam satu jalur.

Pada bagian awal, program memanggil library `iostream` dan `vector` pada [1–2]. Setelah itu, pada [5–7], program mendeklarasikan beberapa variabel utama secara global. Variabel `R` dan `C` digunakan untuk menyimpan jumlah baris dan kolom grid. Variabel `SR` dan `SC` menyimpan posisi awal, sedangkan `FR` dan `FC` menyimpan posisi tujuan. Variabel `totalPath` digunakan untuk menghitung jumlah jalur yang berhasil mencapai titik tujuan.

Pada [9–10], program membuat dua tabel utama, yaitu `grid` dan `visited`. Tabel `grid` digunakan untuk menyimpan kondisi setiap kotak, apakah bisa dilewati atau tidak. Sementara itu, `visited` digunakan untuk menandai apakah suatu kotak sudah pernah dilewati dalam jalur yang sedang dicoba. Bagian ini penting karena program tidak boleh terus-menerus kembali ke kotak yang sama dalam satu jalur. Jika tidak ada penanda seperti `visited`, program dapat berputar tanpa henti, misalnya dari satu kotak ke kotak sebelahnya lalu kembali lagi ke kotak sebelumnya.

Pada [12–13], program menyiapkan dua array, yaitu `dr` dan `dc`, untuk mengatur arah gerakan. Program hanya mengizinkan pergerakan ke empat arah, yaitu atas, bawah, kiri, dan kanan. Array `dr` menunjukkan perubahan baris, sedangkan `dc` menunjukkan perubahan kolom. Misalnya, gerakan ke atas berarti baris dikurangi satu, sedangkan kolom tetap. Dengan adanya dua array ini, program dapat memeriksa semua arah menggunakan satu loop, sehingga alurnya lebih ringkas dan teratur.

Fungsi utama untuk mencari jalur terdapat pada fungsi `dfs` di [15–35]. Fungsi ini menggunakan metode **Depth-First Search (DFS)**, yaitu cara pencarian yang mencoba berjalan sejauh mungkin melalui satu jalur terlebih dahulu sebelum

kembali dan mencoba jalur lain. Secara sederhana, program akan memilih satu arah, terus berjalan selama masih bisa, lalu jika sudah tidak bisa lanjut, program akan mundur ke posisi sebelumnya untuk mencoba kemungkinan arah lainnya. Cara ini cocok digunakan ketika program harus menghitung semua kemungkinan jalur, bukan hanya mencari satu jalur saja.

Terdapat pemeriksaan pada [16–19]. Jika posisi saat ini sama dengan posisi tujuan, maka artinya program sudah berhasil menemukan satu jalur dari titik awal ke titik akhir. Ketika kondisi ini terjadi, nilai `totalPath` ditambah satu. Setelah itu, fungsi langsung berhenti dengan `return`, karena jalur tersebut sudah selesai dihitung. Program kemudian akan kembali ke langkah sebelumnya untuk mencari kemungkinan jalur lain.

Jika posisi saat ini belum mencapai tujuan, maka pada [21] posisi tersebut ditandai sebagai sudah dikunjungi dengan `visited[r][c] = true`. Penandaan ini berarti kotak tersebut sedang digunakan dalam jalur yang sedang dicoba. Tujuannya agar program tidak melewati kotak yang sama dua kali dalam jalur yang sama. Hal ini penting untuk menjaga agar jalur yang dihitung benar-benar jalur yang valid dan tidak berulang-ulang di tempat yang sama.

Setelah posisi ditandai, program memeriksa empat kemungkinan arah melalui perulangan pada [23–31]. Pada setiap perulangan, program menghitung posisi baru yang mungkin dituju menggunakan `nr` dan `nc` pada [24–25]. Variabel `nr` menunjukkan baris baru, sedangkan `nc` menunjukkan kolom baru. Setelah posisi baru dihitung, program memeriksa apakah posisi tersebut masih berada di dalam batas grid pada [27]. Pemeriksaan ini diperlukan agar program tidak mencoba membaca posisi di luar tabel, seperti baris negatif atau kolom yang melebihi ukuran grid.

Jika posisi baru masih berada dalam batas grid, program melanjutkan pemeriksaan pada [28]. Pada bagian ini, program memastikan bahwa kotak tersebut bernilai 0 dan belum pernah dikunjungi dalam jalur yang sedang berjalan. Nilai 0 menunjukkan bahwa kotak dapat dilewati, sedangkan `!visited[nr][nc]` memastikan bahwa kotak tersebut belum dipakai sebelumnya pada jalur yang sama. Jika kedua syarat terpenuhi, maka fungsi `dfs` dipanggil kembali pada [29] untuk melanjutkan pencarian dari posisi baru tersebut.

Pada [34], yaitu `visited[r][c] = false`. Perintah ini digunakan untuk membatalkan tanda kunjungan setelah semua kemungkinan dari posisi tersebut selesai diperiksa. Proses ini disebut sebagai langkah kembali atau **backtracking**. Secara sederhana, ketika program sudah selesai mencoba semua jalan dari suatu kotak, kotak tersebut dibuka kembali agar dapat digunakan pada jalur lain yang berbeda. Tanpa langkah ini, program hanya akan menghitung sebagian jalur saja, karena banyak kotak akan tetap dianggap sudah dikunjungi meskipun sebenarnya jalur sebelumnya sudah selesai diperiksa.

Bagian main dimulai pada [37]. Pada [38], program membaca ukuran grid berupa jumlah baris dan kolom. Setelah itu, pada [40–41], program menyiapkan ukuran grid dan `visited` sesuai dengan jumlah baris dan kolom yang dimasukkan. Semua nilai pada `visited` diatur menjadi `false`, yang berarti pada awalnya belum ada kotak yang dikunjungi. Kemudian pada [43–47], program membaca isi grid satu per satu. Data ini menjadi dasar bagi program untuk mengetahui kotak mana yang dapat dilewati dan kotak mana yang menjadi penghalang.

Pada [49–50], program membaca posisi awal dan posisi tujuan. Setelah semua data yang dibutuhkan tersedia, program memulai pencarian dengan memanggil `dfs(SR, SC)` pada [52]. Pemanggilan ini berarti program mulai mencari semua kemungkinan jalur dari posisi awal. Fungsi `dfs` akan berjalan secara berulang melalui pemanggilan dirinya sendiri sampai semua jalur yang memungkinkan selesai diperiksa.

Setelah proses pencarian selesai, program mencetak nilai `totalPath` pada [54]. Nilai ini menunjukkan jumlah seluruh jalur yang berhasil ditemukan dari titik awal menuju titik tujuan. Jika hasilnya 0, berarti tidak ada jalur yang memungkinkan, misalnya karena titik tujuan terhalang atau seluruh jalan menuju tujuan tertutup. Jika hasilnya lebih dari 0, maka angka tersebut menunjukkan banyaknya pilihan jalur yang dapat ditempuh.

Soal 5: Tree Traversal

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **RedBlack Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTREE terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

```
N
A1 A2 ... AN
Q
T1
T2
...
TQ
```

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTREE.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder] : daftar_angka
[Inorder]  : daftar_angka
[Postorder]: daftar_angka
```


Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Tabel 5 Source Code File RedBlackTree.h

```
1  #ifndef RED_BLACK_TREE_H
2  #define RED_BLACK_TREE_H
3
4  #include <cstdint>
5
6  class RedBlackTree {
7  public:
8      enum class Color { RED, BLACK };
9
10     struct Node {
11         int key;
12         Color color;
13         Node* left;
14         Node* right;
15         Node* parent;
16         bool isNil;
17
18         Node(int value, Color nodeColor, bool
sentinel);
19     };
20
21     private:
22         Node* rootNode;
23         Node* nilNode;
24         std::size_t nodeCount;
25
26     private:
27         void rotateLeft(Node* node);
28         void rotateRight(Node* node);
29         void fixInsert(Node* node);
30         void destroy(Node* node);
31
32     public:
33         RedBlackTree();
34         ~RedBlackTree();
35
36         RedBlackTree(const RedBlackTree& other) =
delete;
37         RedBlackTree& operator=(const RedBlackTree&
other) = delete;
38
39         void insert(int key);
40         bool contains(int key) const;
41
42         bool lowerBound(int key, int& result) const;
43         bool upperBound(int key, int& result) const;
```

44	
45	const Node* root() const;
46	const Node* nil() const;
47	
48	bool empty() const;
49	std::size_t size() const;
50	
51	void clear();
52	};
53	
54	#endif

Tabel 6 Source Code File RedBlackTree.cpp

1	#include "RedBlackTree.h"
2	
3	RedBlackTree::Node::Node(int value, Color
4	nodeColor, bool sentinel)
5	: key(value),
6	color(nodeColor),
7	left(nullptr),
8	right(nullptr),
9	parent(nullptr),
10	isNil(sentinel) {}
11	RedBlackTree::RedBlackTree()
12	: rootNode(nullptr),
13	nilNode(nullptr),
14	nodeCount(0) {
15	nilNode = new Node(0, Color::BLACK, true);
16	nilNode->left = nilNode;
17	nilNode->right = nilNode;
18	nilNode->parent = nilNode;
19	
20	rootNode = nilNode;
21	}
22	
23	RedBlackTree::~~RedBlackTree() {
24	clear();
25	delete nilNode;
26	}
27	
28	void RedBlackTree::rotateLeft(Node* node) {
29	Node* child = node->right;
30	
31	node->right = child->left;
32	if (!child->left->isNil) {

```

33         child->left->parent = node;
34     }
35
36     child->parent = node->parent;
37
38     if (node->parent->isNil) {
39         rootNode = child;
40     } else if (node == node->parent->left) {
41         node->parent->left = child;
42     } else {
43         node->parent->right = child;
44     }
45
46     child->left = node;
47     node->parent = child;
48 }
49
50 void RedBlackTree::rotateRight(Node* node) {
51     Node* child = node->left;
52
53     node->left = child->right;
54     if (!child->right->isNil) {
55         child->right->parent = node;
56     }
57
58     child->parent = node->parent;
59
60     if (node->parent->isNil) {
61         rootNode = child;
62     } else if (node == node->parent->right) {
63         node->parent->right = child;
64     } else {
65         node->parent->left = child;
66     }
67
68     child->right = node;
69     node->parent = child;
70 }
71
72 void RedBlackTree::fixInsert(Node* node) {
73     while (node->parent->color == Color::RED) {
74         Node* parent = node->parent;
75         Node* grandparent = parent->parent;
76
77         if (parent == grandparent->left) {
78             Node* uncle = grandparent->right;
79
80             if (uncle->color == Color::RED) {
81                 parent->color = Color::BLACK;

```

```

82         uncle->color = Color::BLACK;
83         grandparent->color = Color::RED;
84         node = grandparent;
85     } else {
86         if (node == parent->right) {
87             node = parent;
88             rotateLeft(node);
89         }
90
91         node->parent->color = Color::BLACK;
92         node->parent->parent->color =
Color::RED;
93         rotateRight(node->parent->parent);
94     }
95     } else {
96         Node* uncle = grandparent->left;
97
98         if (uncle->color == Color::RED) {
99             parent->color = Color::BLACK;
100             uncle->color = Color::BLACK;
101             grandparent->color = Color::RED;
102             node = grandparent;
103         } else {
104             if (node == parent->left) {
105                 node = parent;
106                 rotateRight(node);
107             }
108
109             node->parent->color = Color::BLACK;
110             node->parent->parent->color =
Color::RED;
111             rotateLeft(node->parent->parent);
112         }
113     }
114 }
115
116 rootNode->color = Color::BLACK;
117 }
118
119 void RedBlackTree::insert(int key) {
120     Node* newNode = new Node(key, Color::RED,
false);
121     newNode->left = nilNode;
122     newNode->right = nilNode;
123     newNode->parent = nilNode;
124
125     Node* parent = nilNode;
126     Node* current = rootNode;
127

```

```

128     while (!current->isNil) {
129         parent = current;
130
131         if (key < current->key) {
132             current = current->left;
133         } else {
134             current = current->right;
135         }
136     }
137
138     newNode->parent = parent;
139
140     if (parent->isNil) {
141         rootNode = newNode;
142     } else if (key < parent->key) {
143         parent->left = newNode;
144     } else {
145         parent->right = newNode;
146     }
147
148     nodeCount++;
149     fixInsert(newNode);
150 }
151
152 bool RedBlackTree::contains(int key) const {
153     Node* current = rootNode;
154
155     while (!current->isNil) {
156         if (key == current->key) {
157             return true;
158         }
159
160         if (key < current->key) {
161             current = current->left;
162         } else {
163             current = current->right;
164         }
165     }
166
167     return false;
168 }
169
170 bool RedBlackTree::lowerBound(int key, int&
result) const {
171     Node* current = rootNode;
172     Node* answer = nilNode;
173
174     while (!current->isNil) {
175         if (current->key >= key) {

```

```

176         answer = current;
177         current = current->left;
178     } else {
179         current = current->right;
180     }
181 }
182
183 if (answer->isNil) {
184     return false;
185 }
186
187 result = answer->key;
188 return true;
189 }
190
191 bool RedBlackTree::upperBound(int key, int&
result) const {
192     Node* current = rootNode;
193     Node* answer = nilNode;
194
195     while (!current->isNil) {
196         if (current->key > key) {
197             answer = current;
198             current = current->left;
199         } else {
200             current = current->right;
201         }
202     }
203
204     if (answer->isNil) {
205         return false;
206     }
207
208     result = answer->key;
209     return true;
210 }
211
212 const RedBlackTree::Node* RedBlackTree::root()
const {
213     return rootNode;
214 }
215
216 const RedBlackTree::Node* RedBlackTree::nil()
const {
217     return nilNode;
218 }
219
220 bool RedBlackTree::empty() const {
221     return nodeCount == 0;

```

222	}
223	
224	std::size_t RedBlackTree::size() const {
225	return nodeCount;
226	}
227	
228	void RedBlackTree::destroy(Node* node) {
229	if (node->isNil) {
230	return;
231	}
232	
233	destroy(node->left);
234	destroy(node->right);
235	delete node;
236	}
237	
238	void RedBlackTree::clear() {
239	destroy(rootNode);
240	rootNode = nilNode;
241	nodeCount = 0;
242	}

Tabel 7 Source Code File prob5.cpp

1	#include <iostream>
2	#include <vector>
3	#include <string>
4	#include "RedBlackTree.h"
5	using namespace std;
6	
	void preorder(const RedBlackTree::Node* node,
	const RedBlackTree::Node* nil, vector<int>&
7	result) {
8	if (node == nil node->isNil) {
9	return;
10	}
11	
12	result.push_back(node->key);
13	preorder(node->left, nil, result);
14	preorder(node->right, nil, result);
15	}
16	
	void inorder(const RedBlackTree::Node* node,
	const RedBlackTree::Node* nil, vector<int>&
17	result) {
18	if (node == nil node->isNil) {
19	return;
20	}
21	

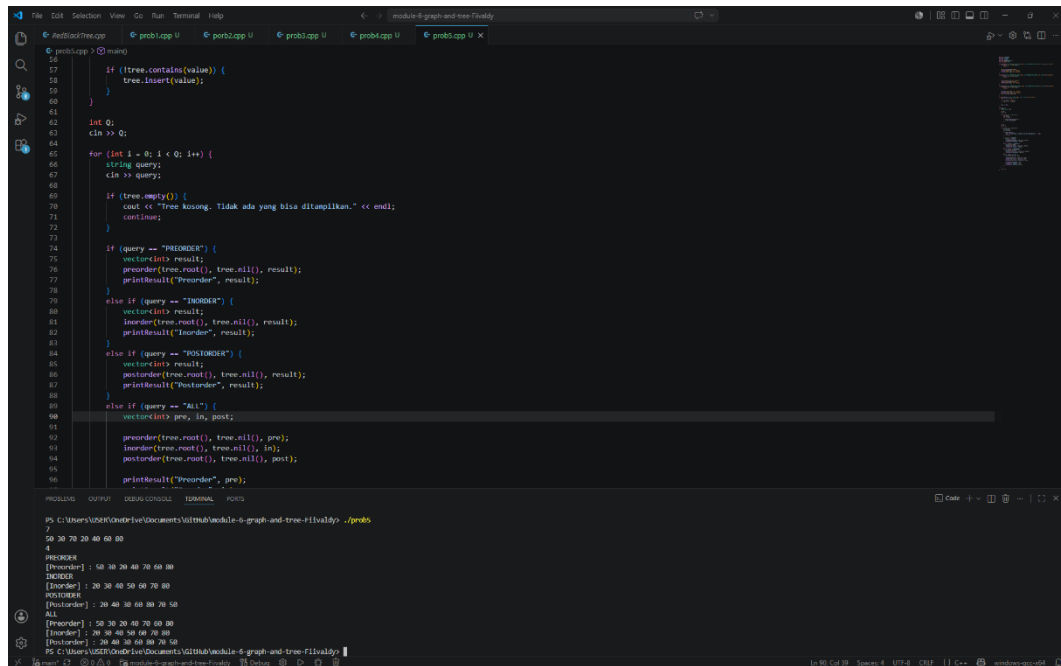

```

22     inorder(node->left, nil, result);
23     result.push_back(node->key);
24     inorder(node->right, nil, result);
25 }
26
27 void postorder(const RedBlackTree::Node* node,
28 const RedBlackTree::Node* nil, vector<int>&
29 result) {
30     if (node == nil || node->isNil) {
31         return;
32     }
33     postorder(node->left, nil, result);
34     postorder(node->right, nil, result);
35     result.push_back(node->key);
36 }
37
38 void printResult(const string& label, const
39 vector<int>& result) {
40     cout << "[" << label << "]" :";
41
42     for (int value : result) {
43         cout << " " << value;
44     }
45     cout << endl;
46 }
47
48 int main() {
49     RedBlackTree tree;
50
51     int N;
52     cin >> N;
53
54     for (int i = 0; i < N; i++) {
55         int value;
56         cin >> value;
57
58         if (!tree.contains(value)) {
59             tree.insert(value);
60         }
61     }
62
63     int Q;
64     cin >> Q;
65
66     for (int i = 0; i < Q; i++) {
67         string query;
68         cin >> query;

```

68	
69	if (tree.empty()) {
70	cout << "Tree kosong. Tidak ada yang
71	bisa ditampilkan." << endl;
72	continue;
73	}
74	if (query == "PREORDER") {
75	vector<int> result;
76	preorder(tree.root(), tree.nil(),
77	result);
78	printResult("Preorder", result);
79	}
80	else if (query == "INORDER") {
81	vector<int> result;
82	inorder(tree.root(), tree.nil(),
83	result);
84	printResult("Inorder", result);
85	}
86	else if (query == "POSTORDER") {
87	vector<int> result;
88	postorder(tree.root(), tree.nil(),
89	result);
90	printResult("Postorder", result);
91	}
92	else if (query == "ALL") {
93	vector<int> pre, in, post;
94	preorder(tree.root(), tree.nil(),
95	pre);
96	inorder(tree.root(), tree.nil(), in);
97	postorder(tree.root(), tree.nil(),
98	post);
99	printResult("Preorder", pre);
100	printResult("Inorder", in);
101	printResult("Postorder", post);
102	}
103	}
	return 0;
	}

B. Output Program



The screenshot shows a C++ program running in a terminal window. The program implements a binary tree with nodes containing values. It performs several operations: inserting values, checking if the tree is empty, and performing preorder, inorder, and postorder traversals. The output shows the results of these operations for a specific set of inputs.

```
PS C:\Users\User\OneDrive\Documents\gitHub\week5-6-graph-and-tree-flivaldy> ./prob5
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Output:

```
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder] : 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 60 30 60 80 70 50
```

Gambar 5 Screenshot Output Program Soal 5

C. Pembahasan

File `RedBlackTree.h` merupakan file header yang berfungsi sebagai rancangan awal dari struktur data **Red-Black Tree**. File ini tidak berisi proses kerja secara lengkap, tetapi hanya berisi daftar komponen, fungsi, dan aturan dasar yang nantinya akan digunakan pada file implementasi, biasanya bernama `RedBlackTree.cpp`. Dengan kata lain, file ini dapat dipahami sebagai “kerangka” atau “daftar isi” dari class `RedBlackTree`, sedangkan isi langkah-langkah detail dari setiap fungsi akan ditulis di file `.cpp`.

Pada bagian [1–2], terdapat perintah `#ifndef`, `#define`, dan pada bagian akhir [49] terdapat `#endif`. Bagian ini digunakan sebagai pelindung agar isi file header tidak dibaca berulang kali oleh compiler. Dalam program C++, file header bisa saja dipanggil oleh beberapa file lain. Jika tidak ada pelindung seperti ini, isi class dapat terbaca lebih dari satu kali dan menyebabkan error. Oleh karena itu, bagian ini penting untuk memastikan bahwa definisi `RedBlackTree` hanya dimasukkan satu kali saat proses kompilasi.

Pada [4], program menyertakan library `<cstdint>`. Library ini digunakan karena program memakai tipe data `std::size_t` pada bagian jumlah node. `std::size_t` biasanya digunakan untuk menyimpan ukuran atau jumlah data karena nilainya tidak bernilai negatif. Dalam konteks Red-Black Tree, tipe ini cocok digunakan untuk menyimpan jumlah node yang ada di dalam tree.

Pada [6], yaitu deklarasi class `RedBlackTree`. Class ini dapat dipahami sebagai wadah yang menyimpan seluruh data dan fungsi yang berhubungan dengan Red-Black Tree. Red-Black Tree sendiri adalah salah satu bentuk binary search tree yang memiliki aturan tambahan berupa warna merah dan hitam pada setiap node. Tujuan dari aturan warna tersebut adalah menjaga agar tree tetap seimbang, sehingga proses pencarian, penambahan, dan beberapa operasi lain dapat berjalan lebih efisien.

Pada [8], terdapat enum class `Color { RED, BLACK };`. Bagian ini digunakan untuk membuat dua pilihan warna pada node, yaitu merah dan hitam. Dalam Red-Black Tree, warna bukan hanya sekadar penanda tampilan, tetapi menjadi bagian dari aturan keseimbangan tree. Warna membantu program menjaga

agar posisi node tidak terlalu berat ke salah satu sisi. Dengan adanya enum class, warna node menjadi lebih jelas karena hanya boleh bernilai RED atau BLACK.

Pada [10–17], program mendefinisikan struktur Node. Node adalah satu bagian atau satu elemen dari tree. Setiap node menyimpan sebuah nilai dan memiliki hubungan dengan node lain. Pada [11], key digunakan untuk menyimpan nilai utama dari node. Nilai inilah yang digunakan saat proses pencarian atau penyisipan data. Pada [12], color digunakan untuk menyimpan warna node, apakah merah atau hitam. Pada [13–15], terdapat pointer left, right, dan parent. Pointer left menunjuk ke anak kiri, right menunjuk ke anak kanan, sedangkan parent menunjuk ke induk dari node tersebut. Dengan tiga hubungan ini, setiap node dapat terhubung membentuk struktur tree.

Pada [16], terdapat variabel isNil. Variabel ini digunakan untuk menandai apakah sebuah node merupakan node khusus bernama nil. Dalam Red-Black Tree, nil biasanya digunakan sebagai penanda ujung atau daun kosong. Jadi, daripada menggunakan nilai kosong biasa, program menyiapkan node khusus untuk mewakili bagian yang tidak memiliki anak. Hal ini membuat pengaturan tree menjadi lebih rapi karena setiap ujung tree tetap memiliki penanda yang jelas.

Pada [18], terdapat constructor untuk Node, yaitu Node(int value, Color nodeColor, bool sentinel);. Constructor ini berfungsi untuk membuat node baru dengan nilai, warna, dan penanda tertentu. Parameter value digunakan untuk mengisi nilai node, nodeColor digunakan untuk menentukan warna node, dan sentinel digunakan untuk menentukan apakah node tersebut merupakan node biasa atau node nil.

Pada [21–23], terdapat tiga data utama yang dimiliki oleh class RedBlackTree, yaitu rootNode, nilNode, dan nodeCount. rootNode adalah penunjuk ke akar tree, yaitu node paling atas dalam struktur tree. Semua pencarian dan penyisipan biasanya dimulai dari root. nilNode adalah node khusus yang digunakan sebagai penanda kosong pada ujung tree. Sementara itu, nodeCount digunakan untuk menyimpan jumlah node yang ada di dalam tree. Ketiga bagian ini dibuat dalam area private, sehingga tidak dapat diakses langsung dari luar class. Hal ini bertujuan agar data utama tree tidak diubah sembarangan dari luar program.

Pada [26–29], terdapat beberapa fungsi private, yaitu `rotateLeft`, `rotateRight`, `fixInsert`, dan `destroy`. Fungsi-fungsi ini hanya digunakan di dalam class karena berhubungan dengan pengaturan internal tree. Fungsi `rotateLeft` dan `rotateRight` digunakan untuk memutar susunan node ke kiri atau ke kanan. Rotasi ini penting pada Red-Black Tree karena ketika node baru dimasukkan, posisi tree bisa menjadi tidak seimbang. Dengan rotasi, susunan tree dapat diperbaiki tanpa merusak aturan urutan nilai. Fungsi `fixInsert` digunakan setelah proses penambahan data untuk memperbaiki aturan warna dan posisi node agar tetap sesuai dengan aturan Red-Black Tree. Sementara itu, `destroy` digunakan untuk menghapus node-node dari memori, biasanya saat tree dibersihkan atau objek dihancurkan.

Pada [32], terdapat constructor `RedBlackTree()`. Fungsi ini akan dijalankan saat objek Red-Black Tree dibuat. Tugas utamanya biasanya adalah menyiapkan tree kosong, membuat node nil, mengatur root awal, dan mengatur jumlah node menjadi nol. Pada [33], terdapat destructor `~RedBlackTree()`. Destructur dijalankan saat objek sudah tidak digunakan lagi. Fungsinya adalah membersihkan memori yang sebelumnya digunakan oleh node-node di dalam tree. Ini penting karena node dibuat menggunakan pointer, sehingga memori harus dibersihkan agar tidak terjadi pemborosan memori.

Pada [35–36], terdapat dua perintah yang menghapus kemampuan penyalinan objek, yaitu copy constructor dan operator assignment. Maksudnya, objek `RedBlackTree` tidak boleh disalin secara langsung menggunakan cara biasa. Hal ini dilakukan karena tree menggunakan banyak pointer yang saling terhubung. Jika tree disalin sembarangan, bisa terjadi masalah seperti dua objek menunjuk ke node yang sama, sehingga saat salah satu objek menghapus node, objek lain bisa ikut rusak. Oleh karena itu, penyalinan dibuat tidak diperbolehkan agar struktur tree tetap aman.

Pada [38], terdapat fungsi `insert(int key)`. Fungsi ini digunakan untuk menambahkan nilai baru ke dalam Red-Black Tree. Nilai yang dimasukkan akan ditempatkan sesuai aturan binary search tree, yaitu nilai yang lebih kecil berada di sebelah kiri dan nilai yang lebih besar berada di sebelah kanan. Setelah nilai dimasukkan, biasanya program akan memanggil proses perbaikan agar aturan Red-Black Tree tetap terjaga.

Pada [39], terdapat fungsi `contains(int key) const`. Fungsi ini digunakan untuk memeriksa apakah suatu nilai terdapat di dalam tree atau tidak. Jika nilai ditemukan, fungsi akan mengembalikan `true`. Jika tidak ditemukan, fungsi akan mengembalikan `false`. Kata `const` menunjukkan bahwa fungsi ini hanya memeriksa data dan tidak mengubah isi tree.

Pada [41–42], terdapat fungsi `lowerBound` dan `upperBound`. Fungsi `lowerBound` digunakan untuk mencari nilai terkecil di dalam tree yang nilainya lebih besar atau sama dengan nilai yang dicari. Sementara itu, `upperBound` digunakan untuk mencari nilai terkecil yang nilainya benar-benar lebih besar dari nilai yang dicari. Hasil pencarian disimpan ke dalam parameter `result`. Kedua fungsi ini berguna ketika program tidak hanya ingin mengetahui apakah sebuah nilai ada, tetapi juga ingin mencari nilai terdekat berdasarkan urutan.

Pada [44–45], terdapat fungsi `root()` dan `nil()`. Fungsi `root()` digunakan untuk mengakses node akar dari tree, sedangkan `nil()` digunakan untuk mengakses node khusus `nil`. Keduanya mengembalikan pointer yang bersifat `const`, sehingga pengguna dapat melihat data tersebut tetapi tidak dapat mengubahnya secara langsung. Hal ini membuat data internal tree tetap aman.

Pada [47–48], terdapat fungsi `empty()` dan `size()`. Fungsi `empty()` digunakan untuk memeriksa apakah tree masih kosong atau sudah memiliki node. Jika tree kosong, fungsi akan mengembalikan `true`. Fungsi `size()` digunakan untuk mengetahui jumlah node yang ada di dalam tree. Kedua fungsi ini membantu pengguna mengetahui kondisi tree tanpa harus memeriksa node satu per satu.

Pada [50], terdapat fungsi `clear()`. Fungsi ini digunakan untuk menghapus seluruh isi tree dan mengembalikannya ke kondisi kosong. Fungsi ini biasanya akan bekerja dengan memanggil fungsi `destroy` untuk menghapus node-node yang ada, lalu mengatur ulang `root` dan jumlah node. Dengan adanya fungsi ini, pengguna dapat mengosongkan tree tanpa harus membuat objek baru.

File **RedBlackTree.cpp** merupakan file implementasi dari rancangan yang sebelumnya sudah dibuat pada **RedBlackTree.h**. Jika file .h berisi daftar fungsi dan bentuk struktur datanya, maka file .cpp ini berisi isi nyata dari fungsi-fungsi tersebut.

Pada bagian awal [1], program memanggil file RedBlackTree.h. Hal ini dilakukan karena semua nama class, struktur Node, warna node, dan daftar fungsi sudah dideklarasikan di dalam file header tersebut. File .cpp ini tidak membuat rancangan baru, tetapi melengkapi isi dari rancangan yang sudah ada. Dengan cara ini, program menjadi lebih rapi karena bagian deklarasi dan bagian implementasi dipisahkan.

Bagian [3–9] berisi constructor untuk Node. Constructor ini digunakan ketika sebuah node baru dibuat. Setiap node memiliki nilai utama yang disimpan pada key, warna yang disimpan pada color, serta hubungan ke node lain melalui left, right, dan parent. Pada awal dibuat, hubungan kiri, kanan, dan induknya masih diisi nullptr karena belum disambungkan ke tree. Selain itu, variabel isNil digunakan untuk membedakan apakah node tersebut adalah node biasa atau node khusus penanda kosong. Node khusus ini penting dalam Red-Black Tree karena ujung-ujung tree tidak dibiarkan benar-benar kosong, tetapi ditandai menggunakan node khusus yang disebut nilNode.

Bagian [11–21] merupakan constructor untuk class RedBlackTree. Fungsi ini berjalan otomatis ketika objek Red-Black Tree dibuat. Pada awalnya, rootNode, nilNode, dan nodeCount disiapkan terlebih dahulu. Nilai nodeCount dibuat 0 karena tree masih kosong. Setelah itu, program membuat nilNode dengan warna hitam. Dalam Red-Black Tree, node kosong atau penanda ujung biasanya dianggap berwarna hitam agar aturan tree tetap terjaga. Kemudian nilNode diarahkan ke dirinya sendiri pada bagian kiri, kanan, dan parent. Hal ini membuat program lebih aman ketika memeriksa node kosong, karena nilNode tetap memiliki alamat yang valid dan tidak menyebabkan kesalahan akses memori. Setelah nilNode siap, rootNode diarahkan ke nilNode, yang berarti tree pada awalnya belum memiliki data.

Bagian [23–26] adalah destructor dari RedBlackTree. Destructor berjalan ketika objek Red-Black Tree sudah tidak digunakan lagi. Di dalamnya, program

memanggil `clear()` untuk menghapus semua node biasa yang ada di dalam tree. Setelah itu, `nilNode` juga dihapus dari memori. Bagian ini penting karena struktur tree menggunakan `new` untuk membuat node, sehingga memori yang digunakan harus dibersihkan secara manual agar tidak terjadi pemborosan memori.

Fungsi `rotateLeft` pada [28–49] digunakan untuk melakukan rotasi ke kiri. Rotasi dapat dipahami sebagai proses mengubah susunan node tanpa mengubah urutan nilai di dalam tree. Pada Red-Black Tree, rotasi diperlukan ketika penambahan node baru membuat susunan tree menjadi kurang seimbang atau melanggar aturan warna. Dalam rotasi kiri, anak kanan dari sebuah node akan naik menggantikan posisi node tersebut, sedangkan node lama akan turun ke sisi kiri dari anak tersebut. Walaupun posisi bentuk tree berubah, urutan nilai tetap benar: nilai yang lebih kecil tetap berada di kiri, dan nilai yang lebih besar tetap berada di kanan. Pada [29], program mengambil anak kanan dari node yang sedang diputar dan menyimpannya dalam variabel `child`. Anak kanan ini nantinya akan naik menggantikan posisi node. Pada [31–34], bagian kiri dari `child` dipindahkan menjadi anak kanan dari node. Jika bagian kiri `child` bukan `nilNode`, maka `parent`-nya diperbarui agar menunjuk ke node. Setelah itu, pada [36–44], program menghubungkan `child` dengan `parent` lama dari node. Jika node sebelumnya adalah root, maka root diganti menjadi `child`. Jika node adalah anak kiri atau anak kanan dari `parent`-nya, maka hubungan tersebut juga disesuaikan. Terakhir, pada [46–47], node dijadikan anak kiri dari `child`, dan `parent` dari node diubah menjadi `child`. Dengan langkah ini, rotasi kiri selesai dilakukan.

Fungsi `rotateRight` pada [51–72] memiliki tujuan yang mirip dengan `rotateLeft`, tetapi arah perubahannya berlawanan. Pada rotasi kanan, anak kiri dari sebuah node akan naik menggantikan posisi node tersebut, sedangkan node lama turun menjadi anak kanan dari anak tersebut. Fungsi ini digunakan ketika struktur tree perlu diperbaiki ke arah kanan. Pada [52], program mengambil anak kiri sebagai `child`. Kemudian pada [54–57], anak kanan dari `child` dipindahkan menjadi anak kiri dari node. Setelah itu, hubungan `parent` diperbaiki pada [59–67], lalu pada [69–70], node dijadikan anak kanan dari `child`. Jadi, rotasi kanan membantu menjaga susunan tree tetap seimbang tanpa merusak aturan urutan nilai.

Fungsi `fixInsert` pada [74–119] merupakan salah satu bagian paling penting dalam Red-Black Tree. Fungsi ini digunakan setelah sebuah node baru dimasukkan ke dalam tree. Dalam Red-Black Tree, node baru biasanya diberi warna merah. Namun, penambahan node merah bisa menyebabkan aturan tree dilanggar, terutama jika parent dari node tersebut juga berwarna merah. Oleh karena itu, fungsi `fixInsert` bertugas memperbaiki warna dan posisi node agar tree kembali memenuhi aturan Red-Black Tree.

Pada [75], program menjalankan perulangan selama parent dari node yang sedang diperiksa berwarna merah. Kondisi ini menunjukkan adanya pelanggaran aturan, karena dalam Red-Black Tree node merah tidak boleh memiliki parent yang juga merah. Pada [76–77], program mengambil parent dan grandparent dari node. Grandparent adalah induk dari parent. Informasi ini dibutuhkan karena perbaikan biasanya melibatkan tiga posisi utama, yaitu node baru, parent, dan grandparent. Pada [79–101], program menangani kondisi ketika parent berada di sisi kiri grandparent. Dalam kondisi ini, program juga mengambil uncle, yaitu saudara dari parent, yang berada di sisi kanan grandparent. Jika uncle berwarna merah, seperti pada [82], maka masalah dapat diselesaikan dengan mengubah warna. Parent dan uncle diubah menjadi hitam, sedangkan grandparent diubah menjadi merah. Setelah itu, node yang sedang diperiksa dinaikkan ke grandparent agar program dapat memeriksa apakah perubahan warna tersebut menyebabkan masalah baru di bagian atas tree.

Jika uncle berwarna hitam, maka perbaikan tidak cukup hanya dengan mengganti warna. Program perlu melakukan rotasi. Pada [88–91], jika node berada di sisi kanan parent, maka program melakukan rotasi kiri terlebih dahulu agar bentuknya menjadi lebih mudah diperbaiki. Setelah itu, pada [94–96], parent diubah menjadi hitam, grandparent diubah menjadi merah, lalu dilakukan rotasi kanan pada grandparent. Langkah ini membuat susunan tree kembali lebih seimbang dan aturan warna dapat dipenuhi.

Bagian [102–115] menangani kondisi sebaliknya, yaitu ketika parent berada di sisi kanan grandparent. Cara kerjanya sama, tetapi arah kiri dan kanan dibalik. Jika uncle berwarna merah, program cukup mengganti warna parent, uncle, dan grandparent. Jika uncle berwarna hitam, program melakukan rotasi kanan atau

rotasi kiri sesuai posisi node. Dengan adanya dua bagian ini, fungsi `fixInsert` dapat memperbaiki tree baik ketika masalah terjadi di sisi kiri maupun sisi kanan.

Pada [118], root selalu diubah menjadi hitam. Ini adalah aturan penting dalam Red-Black Tree, yaitu akar tree harus berwarna hitam. Walaupun proses perbaikan sebelumnya bisa mengubah warna beberapa node, bagian akhir ini memastikan bahwa root tetap sesuai aturan. Dengan demikian, setelah `fixInsert` selesai, tree kembali berada dalam keadaan yang valid.

Fungsi `insert` pada [121–154] digunakan untuk memasukkan nilai baru ke dalam Red-Black Tree. Pada [122], program membuat node baru dengan nilai key, warna merah, dan penanda bahwa node tersebut bukan `nilNode`. Anak kiri, anak kanan, dan parent dari node baru diatur ke `nilNode` pada [123–125]. Warna merah diberikan karena dalam Red-Black Tree, node baru umumnya dimulai sebagai node merah agar tidak langsung menambah jumlah node hitam pada jalur tree.

Pada [127–129], program menyiapkan dua pointer, yaitu `parent` dan `current`. `current` digunakan untuk menelusuri tree dari root, sedangkan `parent` menyimpan node terakhir sebelum posisi kosong ditemukan. Perulangan pada [131–139] digunakan untuk mencari tempat yang tepat bagi node baru. Jika nilai yang dimasukkan lebih kecil dari nilai node saat ini, program bergerak ke kiri. Jika nilainya lebih besar atau sama, program bergerak ke kanan. Cara ini mengikuti aturan binary search tree, yaitu nilai kecil berada di kiri dan nilai besar berada di kanan.

Setelah posisi kosong ditemukan, pada [141] `parent` dari node baru diatur sesuai hasil pencarian. Jika `parent` masih `nilNode`, berarti tree sebelumnya kosong, sehingga node baru menjadi root seperti pada [143–144]. Jika tree tidak kosong, program menentukan apakah node baru menjadi anak kiri atau anak kanan dari `parent` berdasarkan perbandingan nilainya pada [145–149]. Setelah node berhasil dipasang, jumlah node ditambah pada [152], lalu `fixInsert(newNode)` dipanggil pada [153] untuk memperbaiki aturan Red-Black Tree. Jadi, proses `insert` tidak hanya menaruh data baru, tetapi juga menjaga agar tree tetap seimbang.

Fungsi `contains` pada [156–174] digunakan untuk memeriksa apakah suatu nilai terdapat di dalam tree. Program memulai pencarian dari `rootNode`. Selama node yang diperiksa bukan `nilNode`, program membandingkan nilai yang dicari

dengan nilai node saat ini. Jika sama, fungsi langsung mengembalikan true, yang berarti nilai ditemukan. Jika nilai yang dicari lebih kecil, pencarian bergerak ke kiri. Jika lebih besar, pencarian bergerak ke kanan. Jika pencarian akhirnya mencapai nilNode, artinya nilai tidak ditemukan, sehingga fungsi mengembalikan false. Fungsi ini tidak mengubah isi tree, hanya memeriksa keberadaan data.

Fungsi lowerBound pada [176–197] digunakan untuk mencari nilai terkecil di dalam tree yang nilainya lebih besar atau sama dengan nilai yang dicari. Pada awalnya, answer diisi dengan nilNode, yang berarti belum ada jawaban. Selama pencarian berlangsung, jika nilai node saat ini lebih besar atau sama dengan key, maka node tersebut menjadi calon jawaban. Namun, program masih bergerak ke kiri untuk mencari kemungkinan nilai yang lebih kecil tetapi tetap memenuhi syarat. Jika nilai node saat ini lebih kecil dari key, program bergerak ke kanan karena nilai yang dibutuhkan pasti lebih besar. Setelah pencarian selesai, jika answer masih nilNode, berarti tidak ada nilai yang memenuhi syarat. Jika ada, hasilnya disimpan ke dalam result, lalu fungsi mengembalikan true.

Fungsi upperBound pada [199–220] hampir sama dengan lowerBound, tetapi syaratnya sedikit berbeda. Fungsi ini mencari nilai terkecil yang benar-benar lebih besar dari key, bukan sama atau lebih besar. Perbedaannya terlihat pada [204], yaitu kondisi $\text{current} \rightarrow \text{key} > \text{key}$. Jika nilai node lebih besar dari nilai yang dicari, node tersebut menjadi calon jawaban dan pencarian dilanjutkan ke kiri. Jika tidak, pencarian bergerak ke kanan. Fungsi ini berguna ketika program membutuhkan nilai berikutnya setelah suatu nilai tertentu.

Fungsi root pada [222–224] digunakan untuk mengembalikan alamat node root. Fungsi ini memberi akses kepada bagian luar program untuk melihat akar tree. Namun, karena hasilnya bertipe `const Node*`, pengguna hanya dapat melihat data tersebut dan tidak dapat mengubahnya secara langsung. Fungsi nil pada [226–228] memiliki tujuan serupa, yaitu mengembalikan node khusus nilNode. Kedua fungsi ini membantu program lain membaca struktur tree tanpa merusak isi internalnya. Fungsi empty pada [230–232] digunakan untuk memeriksa apakah tree kosong. Jika nodeCount bernilai 0, maka fungsi mengembalikan true. Artinya, belum ada data biasa yang tersimpan di dalam tree. Fungsi size pada [234–236] digunakan untuk mengembalikan jumlah node yang ada di dalam tree. Nilai ini berasal dari

nodeCount, yang bertambah setiap kali data baru berhasil dimasukkan melalui fungsi insert.

Fungsi destroy pada [238–246] digunakan untuk menghapus node dari memori. Fungsi ini bekerja secara bertahap dari suatu node, lalu menghapus anak kiri dan anak kanannya terlebih dahulu. Jika node yang diperiksa adalah nilNode, fungsi langsung berhenti karena nilNode tidak boleh dihapus oleh fungsi ini. Setelah anak kiri dan kanan selesai dihapus, node saat ini baru dihapus dengan delete. Cara ini penting karena jika parent dihapus terlebih dahulu, program bisa kehilangan akses ke anak-anaknya.

Fungsi clear pada [248–252] digunakan untuk menghapus seluruh isi tree. Fungsi ini memanggil destroy(rootNode) untuk menghapus semua node dari root sampai ke bawah. Setelah semua node biasa dihapus, rootNode dikembalikan ke nilNode, dan nodeCount diatur kembali menjadi 0. Dengan demikian, tree kembali ke keadaan kosong seperti saat pertama kali dibuat, tetapi nilNode tetap tersedia untuk digunakan kembali.

File **prob5.cpp** digunakan untuk membuat sebuah **Red-Black Tree**, memasukkan beberapa nilai ke dalam tree, lalu menampilkan isi tree menggunakan beberapa cara penelusuran, yaitu **preorder**, **inorder**, dan **postorder**. Program ini bergantung pada file **RedBlackTree.h**, sehingga proses utama seperti membuat tree, memasukkan data, mengecek data, dan mengambil root sudah disediakan oleh class **RedBlackTree**. File ini lebih berfokus pada cara membaca input, mencegah data yang sama masuk dua kali, serta menampilkan isi tree berdasarkan perintah dari pengguna.

Pada bagian awal [1–4], program memanggil beberapa library dan file pendukung. Library **iostream** digunakan untuk menerima input dan menampilkan output, **vector** digunakan untuk menyimpan hasil penelusuran tree, dan **string** digunakan untuk membaca perintah seperti **PREORDER**, **INORDER**, **POSTORDER**, atau **ALL**. Sementara itu, **RedBlackTree.h** dipanggil agar program dapat menggunakan class **RedBlackTree** yang sudah dibuat sebelumnya. Dengan adanya file header tersebut, program ini tidak perlu menulis ulang cara kerja **Red-Black Tree** dari awal.

Fungsi **preorder** pada [7–15] digunakan untuk menelusuri tree dengan urutan **root**, **kiri**, **kanan**. Maksudnya, program akan membaca nilai node yang sedang dikunjungi terlebih dahulu, lalu melanjutkan ke anak kiri, kemudian ke anak kanan. Pada [8–10], program memeriksa apakah node yang sedang dikunjungi adalah nil atau node kosong. Jika iya, fungsi langsung berhenti agar program tidak mencoba membaca bagian yang sebenarnya tidak berisi data. Jika node tersebut valid, nilainya dimasukkan ke dalam **result** pada [12], lalu program melanjutkan penelusuran ke kiri dan kanan pada [13–14]. Cara ini membuat nilai paling atas dari tree akan muncul lebih dulu dalam hasil.

Fungsi **inorder** pada [17–25] digunakan untuk menelusuri tree dengan urutan **kiri**, **root**, **kanan**. Pada **Red-Black Tree** yang juga mengikuti aturan **Binary Search Tree**, penelusuran **inorder** biasanya menghasilkan data dalam urutan menaik. Program terlebih dahulu menelusuri bagian kiri pada [22], kemudian menyimpan nilai node saat ini pada [23], lalu menelusuri bagian kanan pada [24]. Dengan pola seperti ini, nilai yang lebih kecil akan dikunjungi lebih dahulu,

kemudian nilai tengah, lalu nilai yang lebih besar. Karena itulah, hasil inorder sangat berguna untuk melihat apakah data di dalam tree tersusun dengan benar.

Fungsi postorder pada [27–35] digunakan untuk menelusuri tree dengan urutan **kiri, kanan, root**. Artinya, program akan mengunjungi anak kiri terlebih dahulu, kemudian anak kanan, dan nilai node utama dimasukkan paling akhir. Pada [32–34], urutan tersebut terlihat jelas karena fungsi memanggil bagian kiri, lalu bagian kanan, baru kemudian menyimpan nilai node ke dalam result. Cara penelusuran ini sering digunakan ketika ingin memproses bagian bawah tree terlebih dahulu sebelum node induknya.

Fungsi printResult pada [37–45] digunakan untuk menampilkan hasil penelusuran tree secara rapi. Fungsi ini menerima dua data, yaitu label sebagai nama jenis penelusuran dan result sebagai daftar nilai hasil penelusuran. Pada [38], program menampilkan label seperti [Preorder] :. Setelah itu, pada [40–42], program mencetak seluruh nilai yang ada di dalam result satu per satu. Dengan fungsi ini, program tidak perlu menulis ulang cara mencetak hasil untuk preorder, inorder, dan postorder.

Pada fungsi main di [47]. Pada [48], program membuat objek tree dari class RedBlackTree. Objek inilah yang akan menyimpan seluruh nilai yang dimasukkan pengguna. Setelah itu, pada [50–51], program membaca jumlah data yang akan dimasukkan, yaitu N. Nilai N menentukan berapa banyak angka yang akan dibaca dan dicoba dimasukkan ke dalam tree.

Pada [53–61], program membaca nilai sebanyak N kali. Setiap nilai disimpan sementara dalam variabel value. Sebelum nilai tersebut dimasukkan ke dalam tree, program memeriksa terlebih dahulu apakah nilai tersebut sudah ada menggunakan tree.contains(value) pada [57]. Jika nilai belum ada, barulah program memasukkannya dengan tree.insert(value) pada [58]. Pemeriksaan ini penting karena program ingin menghindari data ganda. Dengan begitu, jika pengguna memasukkan angka yang sama lebih dari satu kali, angka tersebut hanya akan disimpan sekali di dalam tree.

Setelah proses input data selesai, program membaca jumlah perintah pada [64–65]. Jumlah perintah ini disimpan dalam variabel Q. Kemudian pada [67–99], program melakukan perulangan sebanyak Q kali untuk membaca dan menjalankan

setiap perintah. Setiap perintah disimpan dalam variabel query. Perintah inilah yang menentukan jenis penelusuran apa yang akan ditampilkan oleh program.

Sebelum menjalankan perintah, program memeriksa apakah tree kosong pada [71–74]. Jika tree kosong, maka program menampilkan pesan bahwa tidak ada data yang bisa ditampilkan, lalu melanjutkan ke perintah berikutnya. Pemeriksaan ini penting agar program tidak mencoba menelusuri tree yang belum memiliki data. Walaupun Red-Black Tree memiliki nilNode sebagai penanda kosong, dari sisi pengguna tetap perlu diberi pesan yang jelas bahwa tree belum berisi nilai.

Jika input yang dimasukkan adalah PREORDER, maka bagian [77–80] akan dijalankan. Program membuat vector kosong bernama result, kemudian memanggil fungsi preorder dengan root tree sebagai titik awal. Setelah proses penelusuran selesai, hasilnya dicetak menggunakan printResult. Dengan perintah ini, pengguna dapat melihat isi tree dimulai dari node paling atas, kemudian turun ke bagian kiri dan kanan.

Jika input yang dimasukkan adalah INORDER, maka bagian [82–85] akan dijalankan. Program membuat vector result, memanggil fungsi inorder, lalu mencetak hasilnya. Hasil inorder pada tree seperti ini biasanya berbentuk urutan angka dari kecil ke besar. Oleh karena itu, perintah INORDER dapat membantu pengguna melihat data yang tersimpan secara teratur.

Jika input yang dimasukkan adalah POSTORDER, maka bagian [87–90] akan dijalankan. Program menjalankan fungsi postorder, kemudian mencetak hasilnya. Penelusuran ini menunjukkan urutan pembacaan dari bagian bawah tree terlebih dahulu, lalu berakhir pada node paling atas. Dengan begitu, pengguna dapat membandingkan perbedaan pola penelusuran antara preorder, inorder, dan postorder.

Jika input yang dimasukkan adalah ALL, maka bagian [92–99] akan dijalankan. Program membuat tiga vector sekaligus, yaitu pre, in, dan post, untuk menyimpan hasil dari tiga jenis penelusuran. Setelah itu, program menjalankan preorder, inorder, dan postorder secara berurutan, lalu mencetak semuanya. Perintah ALL berguna ketika pengguna ingin melihat seluruh bentuk hasil penelusuran tanpa harus mengetik tiga perintah secara terpisah.

